

Resolución de problemas algorítmicos en C++

Laura Camila Garces Castro

Escuela Colombiana de Ingeniería Julio Garavito

Algoritmos en Sistemas Electrónicos

17 de septiembre de 2026

Resolución de problemas algorítmicos en C++

El presente informe documenta la solución de cinco problemas introductorios de programación propuestos en LeetCode y Exercism. El trabajo se desarrolló en C++17 y reúne ejercicios relacionados con estructuras de datos, recorridos lineales, conversión de bases, evaluación de secuencias y validación de entradas. En cada apartado se explica el problema, la metodología empleada, las principales dificultades, la complejidad computacional y las pruebas realizadas.

El propósito no consiste únicamente en presentar código funcional, sino en justificar las decisiones adoptadas y mostrar de forma organizada cómo se comprobó cada solución. El código fuente completo se encuentra estructurado en un repositorio de GitHub, cuyo enlace deberá incorporarse antes de la entrega definitiva.

Objetivo

Aplicar conceptos básicos de diseño algorítmico y programación en C++ para resolver problemas de distintas plataformas, analizar la eficiencia de las soluciones y documentar el proceso de desarrollo de manera clara y reproducible.

Entorno de desarrollo

Las soluciones se escribieron con el estándar C++17 y pueden compilarse con g++ 11 o una versión posterior. Se utilizaron únicamente recursos de la biblioteca estándar, entre ellos vector, string, unordered_map, algorithm y stdexcept. Git y GitHub se emplean para controlar las versiones y publicar los archivos de la entrega.

Two Sum

Plataforma. LeetCode.

Descripción del problema

El problema solicita encontrar los índices de dos elementos distintos de un arreglo cuya suma sea igual a un valor objetivo. Se asume que existe una única respuesta válida.

Metodología

La solución recorre el arreglo una sola vez. Para cada valor calcula el complemento requerido y consulta si este ya se encuentra en un mapa hash. Cuando el complemento aparece, se devuelven su índice y el índice actual. El elemento actual se almacena después de la consulta para evitar que una posición sea utilizada dos veces.

Dificultades y decisiones de implementación

La dificultad principal fue evitar la comparación de todas las parejas posibles. El uso de unordered_map requiere memoria adicional, pero disminuye el tiempo esperado. También fue necesario insertar cada elemento después de buscar el complemento para resolver correctamente entradas con valores repetidos, como [3, 3].

Código fuente

```
#include <unordered_map>
#include <vector>
using namespace std;

class Solution {
public:
```

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> seen;
    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
        int complement = target - nums[i];
        auto it = seen.find(complement);
        if (it != seen.end()) return {it->second, i};
        seen[nums[i]] = i;
    }
    return {};
}
```

Nota. El código completo y los archivos de prueba se encuentran en el repositorio de la entrega.

Complejidad computacional

El algoritmo tiene tiempo esperado $O(n)$ y espacio adicional $O(n)$.

Pruebas realizadas

Tabla 1

Casos de prueba de Two Sum

Caso	Entrada	Resultado esperado
Básico	[2, 7, 11, 15], objetivo 9	[0, 1]
Valores repetidos	[3, 3], objetivo 6	[0, 1]
Números negativos	[-3, 4, 3, 90], objetivo 0	[0, 2]

Excel Sheet Column Number

Plataforma. LeetCode.

Descripción del problema

Este ejercicio consiste en convertir el título de una columna de Excel, formado por letras mayúsculas, en su número de columna correspondiente. Cada letra representa un valor entre 1 y 26.

Metodología

Se utiliza un acumulador que comienza en cero. Por cada carácter, el resultado previo se multiplica por 26 y se suma el valor de la letra actual, calculado mediante la expresión $\text{letra} - A + 1$. El procedimiento equivale a interpretar la cadena como una representación posicional de base 26 sin dígito cero.

Dificultades y decisiones de implementación

La principal dificultad fue reconocer que no se trata de una base 26 convencional.

La letra A representa 1 y Z representa 26. Se utilizó long long como acumulador intermedio y al final se convirtió el resultado a int, de acuerdo con los límites del ejercicio.

Código fuente

```
#include <string>
using namespace std;

class Solution {
public:
    int titleToNumber(string columnTitle) {
        long long result = 0;
        for (char letter : columnTitle) {
            result = result * 26 + (letter - 'A' + 1);
        }
        return static_cast<int>(result);
    }
};
```

Nota. El código completo y los archivos de prueba se encuentran en el repositorio de la entrega.

Complejidad computacional

El tiempo de ejecución es $O(k)$, donde k es la longitud del título, y el espacio adicional es $O(1)$.

Pruebas realizadas

Tabla 2

Casos de prueba de Excel Sheet Column Number

Caso	Entrada	Resultado esperado
Una letra	A	1
Dos letras	AB	28
Valor amplio	ZY	701

Best Time to Buy and Sell Stock

Plataforma. LeetCode.

Descripción del problema

A partir de una secuencia de precios diarios, se debe calcular la mayor ganancia posible mediante una sola compra y una venta realizada en un día posterior. Si no existe una operación rentable, el resultado debe ser cero.

Metodología

El primer precio se conserva como mínimo histórico. Durante el recorrido, cada precio se considera una posible venta y se calcula la diferencia frente al mínimo observado. La mejor ganancia y el mínimo histórico se actualizan cuando corresponde.

Dificultades y decisiones de implementación

La principal consideración fue conservar el orden temporal. No es suficiente localizar el valor mínimo y el máximo globales porque la compra debe ocurrir antes de la venta. El algoritmo solo emplea información correspondiente a los días ya recorridos.

Código fuente

```
#include <algorithm>
#include <vector>
using namespace std;

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int minimumPrice = prices[0];
        int bestProfit = 0;
        for (int price : prices) {
            bestProfit = max(bestProfit, price - minimumPrice);
            minimumPrice = min(minimumPrice, price);
        }
        return bestProfit;
    }
};
```

Nota. El código completo y los archivos de prueba se encuentran en el repositorio de la entrega.

Complejidad computacional

El algoritmo se ejecuta en $O(n)$ y utiliza $O(1)$ de espacio adicional.

Pruebas realizadas

Tabla 3

Casos de prueba de Best Time to Buy and Sell Stock

Caso	Entrada	Resultado esperado
------	---------	--------------------

Existe ganancia	[7, 1, 5, 3, 6, 4]	5
Secuencia descendente	[7, 6, 4, 3, 1]	0
Dos días	[1, 2]	1

Armstrong Numbers

Plataforma. Exercism.

Descripción del problema

El ejercicio solicita determinar si un entero no negativo es un número de Armstrong. Para ello, cada dígito se eleva a la cantidad total de dígitos y posteriormente se suman todos los resultados.

Metodología

El número se convierte en una cadena para determinar su cantidad de dígitos. Cada carácter se transforma nuevamente en un valor entero y se eleva al exponente correspondiente mediante una función auxiliar de potencia entera. Finalmente, la suma se compara con el número original.

Dificultades y decisiones de implementación

Se evitó utilizar pow porque trabaja con punto flotante y podría introducir conversiones innecesarias. En su lugar, se implementó una potencia basada en multiplicaciones enteras repetidas. La solución no requiere librerías externas.

Código fuente

```
#include "armstrong_numbers.h"
#include <string>

namespace armstrong_numbers {
static int integer_power(int base, int exponent) {
    int result = 1;
    for (int i = 0; i < exponent; ++i) result *= base;
    return result;
}

bool is_armstrong_number(int number) {
    const std::string digits = std::to_string(number);
    const int exponent = static_cast<int>(digits.size());
    int sum = 0;
    for (char character : digits) {
        sum += integer_power(character - '0', exponent);
    }
}
```

```
}  
    return sum == number;  
}  
}
```

Nota. El código completo y los archivos de prueba se encuentran en el repositorio de la entrega.

Complejidad computacional

Con la función de potencia repetida, el tiempo es $O(d^2)$, donde d representa el número de dígitos. El espacio adicional es $O(d)$ por la conversión a cadena.

Pruebas realizadas

Tabla 4

Casos de prueba de Armstrong Numbers

Caso	Entrada	Resultado esperado
Un dígito	9	true
Número Armstrong	153	true
No Armstrong	154	false

Hamming Distance

Plataforma. Exercism.

Descripción del problema

La distancia de Hamming corresponde al número de posiciones diferentes entre dos cadenas de ADN de igual longitud. Cuando las longitudes son distintas, la operación no se encuentra definida.

Metodología

Primero se comparan las longitudes de las cadenas. Si son diferentes, se lanza una excepción de tipo `std::domain_error`. Si son iguales, ambas cadenas se recorren mediante un índice y se incrementa un contador por cada posición cuyo carácter sea diferente.

Dificultades y decisiones de implementación

La validación debe realizarse antes del recorrido para respetar la definición del problema y evitar accesos inválidos. La interfaz propuesta por Exercism requiere que las longitudes diferentes produzcan una excepción.

Código fuente

```
#include "hamming.h"
#include <stdexcept>

namespace hamming {
int compute(const std::string& first, const std::string& second) {
    if (first.size() != second.size()) {
        throw std::domain_error("DNA strands must have equal length");
    }
    int distance = 0;
    for (std::size_t i = 0; i < first.size(); ++i) {
        if (first[i] != second[i]) ++distance;
    }
    return distance;
}
}
```

Nota. El código completo y los archivos de prueba se encuentran en el repositorio de la entrega.

Complejidad computacional

El procedimiento requiere $O(n)$ de tiempo y $O(1)$ de espacio adicional.

Pruebas realizadas

Tabla 5

Casos de prueba de Hamming Distance

Caso	Entrada	Resultado esperado
Cadenas iguales	GATTACA / GATTACA	0
Cadenas diferentes	Ejemplo completo de Exercism	7
Longitud desigual	A / AG	Excepción

Repositorio y reproducibilidad

Los archivos se organizaron en carpetas separadas para LeetCode y Exercism. La raíz del proyecto incluye un archivo README con instrucciones y un script denominado run_tests.sh que permite compilar y ejecutar las pruebas locales. La carpeta informe contiene la versión editable y la versión final en PDF.

Enlace de la rama de GitHub: [PEGAR AQUÍ EL ENLACE DEFINITIVO].

Antes de entregar, se debe verificar que el repositorio sea accesible para el docente, que la rama incluya las iniciales del integrante y que todos los archivos del código fuente puedan consultarse sin solicitar permisos adicionales.

Conclusiones

Las soluciones desarrolladas demuestran que la elección de una estrategia adecuada permite resolver problemas introductorios con recorridos eficientes. El mapa hash utilizado en Two Sum reduce las comparaciones; el seguimiento del mínimo histórico resuelve el problema de compra y venta en un solo recorrido; y los ejercicios restantes procesan los caracteres o dígitos de manera directa.

También se comprobó que la corrección de un algoritmo depende de considerar casos especiales. Los valores repetidos, las secuencias descendentes, la aritmética entera y las cadenas con longitudes distintas requieren validaciones específicas que deben explicarse junto con el código.

El uso de Git y GitHub mejora la reproducibilidad del trabajo porque permite conservar el historial de cambios, organizar las soluciones y facilitar la evaluación.